

100 exercices pour maîtriser le C

Parcours progressif en quatre niveaux : **Débutant** → **Intermédiaire** → **Confirmé** → **Maîtrise**.
Chaque exercice est volontairement court à énoncer et te laisse concevoir la solution toi-même.

Mode d'emploi

Compilation systématique avec les avertissements activés :

```
bash

gcc -Wall -Wextra -g -std=c11 exo.c -o exo && ./exo
```

À partir du niveau 3 (allocation dynamique), ajoute une vérification mémoire :

```
bash

valgrind --leak-check=full ./exo      # sur Fedora
leaks --atExit -- ./exo               # sur macOS
```

Boucle de correction : fais un exercice, puis envoie-moi ton code en précisant le numéro (« voici ma réponse à l'exercice E42 »). Je te corrige comme un professeur : ce qui est juste, ce qui est faux et pourquoi, et les pistes d'amélioration (lisibilité, sécurité mémoire, idiomes C, complexité). Pas de solution toute faite tant que tu n'as pas essayé.

Règle du jeu : ne saute pas de niveau. Chaque bloc suppose le précédent acquis.

Niveau 1 — Débutant (E1 à E25)

Objectif : types, entrées/sorties, opérateurs, conditions, boucles.

Bases et entrées/sorties

- **E1.** Affiche (`Hello, C!`) suivi d'un retour à la ligne.
- **E2.** Déclare un (`int`), un (`float`), un (`char`) et affiche-les avec les bons spécificateurs (`%d`), (`%f`), (`%c`).
- **E3.** Lis deux entiers au clavier (`scanf`) et affiche leur somme, différence, produit, quotient et reste.
- **E4.** Convertis une température saisie en Celsius vers Fahrenheit.
- **E5.** Calcule le périmètre et l'aire d'un rectangle à partir de sa largeur et sa hauteur.
- **E6.** Calcule l'aire d'un cercle (utilise (`const double PI = 3.14159;`)).
- **E7.** Échange le contenu de deux variables (`int`) à l'aide d'une variable temporaire, et affiche avant/après.
- **E8.** Lis trois notes (`float`) et affiche leur moyenne.
- **E9.** Convertis un nombre de secondes saisi en heures, minutes et secondes.

Conditions

- **E10.** Indique si un entier saisi est pair ou impair.
- **E11.** Affiche le plus grand de deux nombres.
- **E12.** Affiche le plus grand de trois nombres.
- **E13.** Indique si un nombre est positif, négatif ou nul.
- **E14.** Détermine si une année saisie est bissextile.
- **E15.** À partir d'un âge saisi, affiche la catégorie : enfant (<12), ado (12-17), adulte (18-64), senior (≥65). Utilise un (`switch`) ou des (`else if`).
- **E16.** Écris une calculatrice simple : deux nombres + un opérateur (`+ - * /`) lu en (`char`), traité par un (`switch`).

Boucles

- **E17.** Affiche la table de multiplication d'un nombre saisi (de ×1 à ×10).
- **E18.** Calcule la somme des entiers de 1 à N.
- **E19.** Calcule la factorielle de N avec une boucle.
- **E20.** Affiche les nombres de 1 à 100, mais (`Fizz`) pour les multiples de 3, (`Buzz`) pour 5, (`FizzBuzz`) pour 15.
- **E21.** Affiche la table de multiplication complète de 1 à 9 (boucles imbriquées).
- **E22.** Indique si un nombre saisi est premier.
- **E23.** Calcule (`x`) puissance (`n`) sans utiliser (`pow()`).
- **E24.** Inverse les chiffres d'un entier : (`12345`) → (`54321`).
- **E25.** Affiche les N premiers termes de la suite de Fibonacci.

Niveau 2 — Intermédiaire (E26 à E60)

Objectif : fonctions, récursion, tableaux, matrices, chaînes de caractères.

Fonctions

- **E26.** Écris une fonction `int est_pair(int n)` qui renvoie 1 ou 0.
- **E27.** Écris `int max2(int a, int b)` puis `int max3(int a, int b, int c)` en réutilisant la première.
- **E28.** Écris `unsigned long factorielle(int n)` (version itérative).
- **E29.** Écris `int est_premier(int n)` renvoyant un booléen.
- **E30.** Écris `int pgcd(int a, int b)` par l'algorithme d'Euclide (boucle).
- **E31.** Écris un petit module de conversion : `celsius_to_f`, `f_to_celsius`, et teste-les dans `main`.

Récursion

- **E32.** Réécris la factorielle de façon récursive.
- **E33.** Réécris Fibonacci de façon récursive. Compare la lenteur avec la version itérative pour N grand.
- **E34.** Écris une fonction récursive qui calcule la somme des chiffres d'un entier.
- **E35.** Écris une puissance `x^n` récursive.
- **E36.** Écris le PGCD d'Euclide en version récursive.

Tableaux

- **E37.** Remplis un tableau de 10 entiers saisis, puis affiche-les.
- **E38.** Calcule la somme et la moyenne des éléments d'un tableau.
- **E39.** Trouve le minimum et le maximum d'un tableau.
- **E40.** Recherche linéaire : indique l'indice d'une valeur, ou -1 si absente.
- **E41.** Inverse un tableau **sur place** (sans tableau auxiliaire).
- **E42.** Compte le nombre d'occurrences d'une valeur dans un tableau.
- **E43.** Trie un tableau par tri à bulles.
- **E44.** Trie un tableau par tri par sélection.
- **E45.** Recherche dichotomique dans un tableau **trié**.
- **E46.** Fusionne deux tableaux triés en un seul tableau trié.
- **E47.** Supprime les doublons d'un tableau.

Matrices (tableaux 2D)

- **E48.** Additionne deux matrices de même taille.
- **E49.** Calcule la transposée d'une matrice.
- **E50.** Multiplie deux matrices.

Chaînes de caractères

- **E51.** Calcule la longueur d'une chaîne **sans** `(strlen)`.
 - **E52.** Copie une chaîne dans une autre **sans** `(strcpy)`.
 - **E53.** Concatène deux chaînes.
 - **E54.** Compte les voyelles et les consonnes d'une phrase.
 - **E55.** Inverse une chaîne.
 - **E56.** Vérifie si un mot est un palindrome.
 - **E57.** Compte le nombre de mots d'une phrase.
 - **E58.** Convertis une chaîne en majuscules, puis en minuscules.
 - **E59.** Compare deux chaînes **sans** `(strcmp)` (renvoie <0, 0, >0).
 - **E60.** Recherche une sous-chaîne dans une chaîne (renvoie l'indice de départ ou -1).
-

Niveau 3 — Confirmé (E61 à E85)

Objectif : pointeurs, allocation dynamique, structures, listes chaînées, fichiers, bits. À partir d'ici, vérifie chaque programme avec Valgrind.

Pointeurs

- **E61.** Écris `(void swap(int *a, int *b))` qui échange deux entiers via pointeurs.
- **E62.** Écris une fonction qui double une valeur en la modifiant directement via un pointeur.
- **E63.** Parcourt et affiche un tableau en utilisant **uniquement** l'arithmétique de pointeurs (pas d'indice `([])`).
- **E64.** Écris `(void min_max(const int *t, int n, int *min, int *max))` qui renvoie les deux résultats par pointeurs.

Allocation dynamique

- **E65.** Alloue un tableau de N entiers avec `(malloc)`, remplis-le, affiche-le, puis libère-le.
- **E66.** Pars d'un petit tableau et agrandis-le avec `(realloc)` quand il est plein.
- **E67.** Demande N à l'exécution, alloue le tableau correspondant, lis N valeurs et calcule leur moyenne.
- **E68.** Alloue dynamiquement une matrice N×M (tableau de pointeurs), remplis-la et libère-la proprement.
- **E69.** Écris volontairement un programme avec une fuite mémoire, observe-la dans Valgrind, puis corrige-la.

Structures

- **E70.** Définis `struct Point { double x, y; }` et écris une fonction qui calcule la distance entre deux points.
- **E71.** Définis `struct Etudiant` (nom, âge, moyenne). Crée un tableau de 5 étudiants et affiche-les.
- **E72.** Trie le tableau d'étudiants par moyenne décroissante.
- **E73.** Écris une fonction qui modifie un `struct Etudiant` en le recevant par pointeur (`Etudiant *`).
- **E74.** Utilise `typedef` pour simplifier l'écriture, et imbrique une structure dans une autre (`struct Date` dans `struct Etudiant`).

Listes chaînées

- **E75.** Définis un nœud de liste chaînée simple et écris une fonction d'ajout **en tête**.
- **E76.** Écris une fonction qui parcourt et affiche la liste.
- **E77.** Écris une fonction d'ajout **en queue**.
- **E78.** Supprime un nœud contenant une valeur donnée.
- **E79.** Inverse une liste chaînée.
- **E80.** Compte les éléments, puis écris une fonction qui libère toute la liste (vérifie : zéro fuite dans Valgrind).

Fichiers

- **E81.** Écris plusieurs lignes de texte dans un fichier.
- **E82.** Lis et affiche un fichier ligne par ligne.
- **E83.** Mini `wc` : compte les lignes, mots et caractères d'un fichier.
- **E84.** Copie le contenu d'un fichier vers un autre.

Manipulation de bits

- **E85.** Implémente un système de droits par drapeaux (`READ`, `WRITE`, `EXEC`) : fonctions pour activer (`|`), tester (`&`), retirer (`& ~`) et basculer (`^`) un droit.

Niveau 4 — Maîtrise (E86 à E100)

Objectif : algorithmes, structures de données, et mini-projets de synthèse.

Algorithmes

- **E86.** Implémente le tri rapide (quicksort) récursif.
- **E87.** Implémente le tri fusion (merge sort).
- **E88.** Implémente une **pile** (LIFO) : `push`, `pop`, `peek`, `is_empty`.
- **E89.** Implémente une **file** (FIFO) : `enqueue`, `dequeue`.
- **E90.** Implémente une table de hachage simple (chaînage) avec `insert` et `get`.
- **E91.** Implémente un arbre binaire de recherche : insertion et parcours infixe (in-order).

Mini-projets

- **E92.** Évaluateur d'expressions arithmétiques en notation postfixée (RPN) à l'aide d'une pile.
- **E93.** Parseur de fichier CSV : lis un fichier et affiche les colonnes proprement alignées.
- **E94.** Réécris ta propre version de `strtok` (découpe une chaîne selon des séparateurs).
- **E95.** Buffer circulaire (ring buffer) de taille fixe : écriture/lecture avec gestion du dépassement.
- **E96.** Compteur de fréquence de mots : lis un texte et affiche les mots les plus fréquents (réutilise ta table de hachage E90).
- **E97.** Jeu du « plus ou moins » : le programme tire un nombre (`rand`), l'utilisateur devine, retour haut/bas.
- **E98.** Mini interpréteur de commandes : lis une ligne, découpe-la et exécute une action selon le premier mot (`add`, `list`, `quit`...).
- **E99.** Gestionnaire de contacts (CRUD) : ajouter, lister, rechercher, supprimer des contacts en mémoire (tableau dynamique de structures).
- **E100. Projet de synthèse — gestion d'inventaire** : menu interactif, structures, allocation dynamique, persistance dans un fichier (sauvegarde/chargement), zéro fuite mémoire vérifiée à Valgrind. C'est l'exercice qui mobilise tout le reste.

Ressources d'appoint

- **Exercism (track C)** — exercices corrigés avec mentorat humain.
- **PYnative — C exercises** — plus de 320 problèmes thématiques avec solutions (pour t'auto-corriger quand je ne suis pas là).
- **w3resource — C programming exercises** — large banque débutant/intermédiaire.
- **Compiler Explorer (godbolt.org)** — pour voir l'assembleur généré et comprendre ce que fait vraiment le compilateur.
- **Valgrind** — ton meilleur professeur sur la gestion mémoire à partir du niveau 3.

Conseil : tiens un dépôt Git (un dossier par niveau, un fichier par exercice). Tu auras une trace de ta progression, et c'est exactement le genre de portfolio qui parle à un jury de cycle ingénieur.